

1.Explain hill climbing algorithm with suitable example.

<https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/>

2.Describe alpha-beta pruning and its significance.

Alpha–Beta Pruning (in Game Playing AI)

♦ What is Alpha–Beta Pruning?

Alpha–Beta Pruning is an **optimization technique of the Minimax algorithm** used in game playing (like chess, tic-tac-toe).

👉 It **removes (prunes)** branches of the game tree that **cannot affect the final decision**, reducing computation.

♦ Key Idea

Instead of exploring all possible moves:

- Skip those moves that are **already worse than a previously examined move**
-

♦ Important Terms

- **Alpha (α)**
→ Best value that the **MAX player** can guarantee so far
- **Beta (β)**
→ Best value that the **MIN player** can guarantee so far

👉 Condition for pruning:

If $\alpha \geq \beta$, stop exploring that branch (cut it off)

♦ Working (Step-by-Step)

1. Start with Minimax tree
2. Traverse depth-first
3. Update:
 - α for MAX nodes
 - β for MIN nodes
4. If at any point:
 - $\alpha \geq \beta \rightarrow$ **prune remaining branches**

♦ Example (Simple)

Consider a game tree:

- MAX node at root
- MIN nodes below

Step:

- MAX explores left subtree → gets value **10**
→ $\alpha = 10$
- Now exploring right subtree (MIN node):
 - First child gives value **5** → $\beta = 5$

👉 Since:

$$\alpha (10) \geq \beta (5)$$

✗ No need to check other children of this MIN node

➡ **Branch is pruned**

♦ Why is it Important? (Significance)

- ✓ Reduces computation time
 - ✓ Avoids unnecessary node evaluation
 - ✓ Allows deeper search in same time
 - ✓ Improves performance of Minimax
-

♦ Complexity

- Without pruning:
👉 $O(b^d)$
- With Alpha-Beta Pruning:
👉 $O(b^{d/2})$ (best case)

(b = branching factor, d = depth)

♦ Advantages

- ✓ Faster decision making
- ✓ Same result as Minimax (no loss of accuracy)
- ✓ Efficient for large game trees

♦ Limitations

- ✗ Performance depends on node ordering
- ✗ Still expensive for very large trees

3. Describe local search algorithms with examples.

Local Search Algorithms (AI)

♦ What are Local Search Algorithms?

Local Search Algorithms are techniques used in AI to find **optimal or near-optimal solutions** by exploring **neighboring states** instead of searching the entire space.

👉 They focus on:

- Current solution (state)
- Move to a better nearby solution

♦ Simple Definition (Exam Ready)

Local search algorithms start with an initial solution and iteratively improve it by moving to neighboring states until an optimal or satisfactory solution is found.

♦ Key Characteristics

- ✓ Do not store full search tree
- ✓ Use very less memory
- ✓ Suitable for optimization problems
- ✓ Work on a single current state

♦ General Working

1. Start with an initial solution
2. Evaluate it
3. Move to a better neighbor
4. Repeat until no improvement

◆ Types of Local Search Algorithms

1. Hill Climbing

- Moves to the **best neighboring state**
- Stops when no better neighbor exists

👉 Example:

Climbing a hill step-by-step to reach the peak

2. Simulated Annealing

- Allows **sometimes worse moves** to escape local maxima
- Inspired by heating & cooling process

👉 Example:

A metal cooling slowly to reach minimum energy state

3. Genetic Algorithm

- Works on a **population of solutions**
- Uses:
 - Selection
 - Crossover
 - Mutation

👉 Example:

Evolution process (survival of the fittest)

◆ Example (Easy)

🧩 8-Queens Problem

Goal: Place 8 queens on chessboard without attacking each other

👉 Local search:

- Start with random placement
- Move queens to reduce conflicts
- Repeat until solution found

◆ Advantages

- ✓ Memory efficient
- ✓ Fast for large problems
- ✓ Easy to implement

◆ Disadvantages

✗ Can get stuck in:

- Local maxima
- Plateau
- Ridge

✗ Not guaranteed optimal solution

◆ Applications

- Game playing (Chess, Tic-Tac-Toe)
 - Optimization problems
 - Scheduling
 - Machine learning
-